ARDMS EMERGENCY RESCUE SYSTEM

MOBILE ARCHITECTURE & REAL-TIME CONNECTIVITY GATEWAY

Section 4 & 5 | Operational Synchronization Ledger

4. MOBILE APPLICATION ARCHITECTURE & INTEGRATION

The ARDMS Emergency Rescue Suite is engineered using React Native (bare Expo workflow) to operate reliably in high-performance, offline-first environments. The system utilizes a distributed, real-time client-server architecture. All mission components (HQ Web Command Center, Rescue Officer App, and Public SOS System) communicate over bidirectional WebSocket channels and REST APIs to synchronize maps, coordinates, dispatches, and status states with sub-second latencies.

ARDMS-Rescue Officer App:

- Platform: React Native (Bare Expo SDK 54) with custom WebView wrapper.
- Live Telemetry: Tracks coordinates in real-time, streaming locations to HQ.
- Group Polygons: Displays group boundary limits on offline-capable Leaflet maps.

ARDMS-Public Support System:

- Platform: React Native WebView wrapper for public submissions.
- Location Fetching: Automatically resolves exact GPS coordinates on launch.
- Quick SOS: Actionable panic buttons for immediate grid dispatching.

2. BACKEND ARCHITECTURE & SERVER STACK

Core Stack & Technologies:

- **Runtime:** Node.js LTS (High Performance Async I/O)
- **Framework:** Express.js v5 (Robust REST API Routing)
- **Database:** SQLite3 with Write-Ahead Logging (WAL) Mode Enabled
- **Real-time:** WebSockets (ws protocol) on Port 3001 with Auto Heartbeat
- **Security:** JSON Web Tokens (JWT) + Cryptographic Salted bcryptjs
- **Management:** PM2 Process Daemonization with Zero-Downtime Auto-Restart
- **Socket Handshake (REGISTER):** Clients segregate into specific channels by transmitting a "REGISTER" message packet upon websocket connection (e.g. room: "rescuers" or "admin").
- **Active Heartbeats (PING/PONG):** A bidirectional keep-alive timer sends PING frames from server to clients every 15 seconds. If a client fails to reply with PONG within 5 seconds, the server terminates the socket to free resources.
- **Offline Telemetry Buffer:** If reception drops, client coordinates and actions queue up locally. On the backend, pending task assignments and dispatches are serialized and buffered in memory.

5. REAL-TIME GATEWAY & RECONNECT TECHNOLOGY

To safeguard operation data in low-reception zones, the ARDMS connection gateway utilizes specialized sync and reconnection architectures:

- **Auto-Flush Sync Queue:** The instant a disconnected client registers back online, the gateway identifies the client, restores the session, and automatically flushes the queue.
- **Out-of-Band State Pulling:** Frontends initiate a silent HTTPS API pull upon socket reconnection to refresh Leaflet maps and sync local states with the database.

3. ARDMS WEB COMMAND CENTER

The primary interface for operational command and control is a highly polished, responsive single-page web dashboard designed with modern glassmorphic aesthetics. Features include:

- **Core Client UI:** HTML5, Modern Vanilla CSS (Sleek dark panel aesthetics)
- **Maps Engine:** Leaflet.js API (Marker clustering, dynamic tactical routing)
- **Pulsing Polyline:** Dynamic drawing connecting Officer & SOS coordinates
- **Group Boundaries:** Dashed blue polygons representing squad perimeter limits
- **Digital Clock:** Real-time AM/PM ticking digital clock inside the navbar

5.5. AUTONOMOUS TASK MANAGEMENT (AI SYSTEM)

To eliminate manual delays during high-stress disaster events, the system introduces fully autonomous triage and task distribution powered by advanced AI:

- **AI Auto-Assignment:** An automated backend process powered by Google Gemini 1.5 Pro AI. It intelligently processes live situational data, interpreting JSON telemetry payloads to assign new incoming SOS requests to the most appropriate rescuer. The AI evaluates real-time GPS proximity, rescuer workload, traffic estimations, and unit type suitability.
- **Dynamic Reassignment Buffer:** The system incorporates a strict fail-safe timeout logic. If an assigned rescuer loses internet connectivity, ignores the task beyond the configured interval, or manually declines the dispatch, the assignment is instantly revoked and auto-routes the task to the next optimal candidate.

WebViews are forced to bypass aggressive localized caching, guaranteeing absolute synchronization of task states across all units.

6. WINDOWS HOST & PRIVATE DEVICE TESTING GUIDE

To run local testing on physical smartphones and private devices over local Wi-Fi, execute the following step-by-step Windows deployment guide:

Prerequisite Setup (Windows Host Firewall):

Before physical mobile devices can connect to the Windows machine, local port traffic must be allowed through the Windows Defender Firewall. Run the provided Fix_Firewall.bat as Administrator. This adds two inbound netsh rules: one for TCP Port 3001 and one allowing node.exe through the system firewall.

Step-by-Step Localhost & Mobile Synchronization:

1. **Establish Shared Wi-Fi:** Connect both your host Windows laptop and your private mobile device (Android/iOS) to the exact same local Wi-Fi network.
2. **Execute Auto-Synchronizer:** Run "python sync_apps.py" in the workspace root. The script auto-resolves your active Windows LAN IP (e.g., 192.168.1.5), patches it across all HTML previews and App.js configurations, and compiles WebViews.
3. **Launch Server with PM2:** Start the Express backend under process monitoring: inside "system-backend/", run "pm2 start server.js --name "ardms-backend" --watch" in powershell.
4. **Deploy APK on Mobile Device:** Copy and install the pre-compiled APK file ("public-sos-app-release.apk" or "rescuer-app-release.apk" inside "/APKs") to your private Android handset.
5. **Open App and Verify:** Launch the application on your phone. It will immediately connect to your laptop's LAN IP, syncing live coordinates, markers, maps, and commands instantly!